

Planted motif search

From Wikipedia, the free encyclopedia

In the field of computational biology, a **planted motif search** (PMS) also known as a ***(l, d)*-motif search** (LDMS) is a method for identifying conserved motifs within a set of nucleic acid or peptide sequences.

PMS is known to be NP-complete. The time complexities of most of the planted motif search algorithms depend exponentially on the alphabet size and *l*. The PMS problem was first introduced by Keich and Pevzner.^[1]

The problem of identifying meaningful patterns (e.g., motifs) from biological data has been studied extensively since they play a vital role in understanding gene function, human disease, and may serve as therapeutic drug targets.

Contents

- 1 Description
- 2 Notation
- 3 Algorithms
 - 3.1 Approximate
 - 3.1.1 Random projection
 - 3.1.2 Pattern branching
 - 3.2 Exact
 - 3.2.1 WINNOWER and SP-STAR
 - 3.2.2 PMS series
 - 3.2.2.1 PMS0
 - 3.2.2.2 PMS1
 - 3.2.2.3 PMS2
 - 3.2.2.4 PMS3
 - 3.2.2.5 PMSPRune
 - 3.2.2.6 PMS4
 - 3.2.2.7 PMS5 and PMS6
 - 3.2.2.8 qPMSPRune and qPMS7
 - 3.2.3 RISOTTO
 - 3.3 Relative performance
- 4 References
- 5 External links

Description

The search problem may be summarized as follows:

Input are n strings $(s_1, s_2, \dots, s_n)$ of length m each from an alphabet Σ and two integers l and d . Find all strings x such that $|x| = l$ and every input string contains at least one variant of x at a Hamming distance of at most d . Each such x is referred to as an (l, d) motif.

For example, if the input strings are GCGCGAT, CACGTGA, and CGGTGCC; $l = 3$ and $d = 1$, then GGT is a motif of interest. Note that the first input string has GAT as a substring, the second input string has CGT as a substring, and the third input string has GGT as a substring. GAT is a variant of GGT that is within a Hamming distance of 1 from GGT, etc. Call the variants of a motif that occur in the input strings as instances of the motif. For example, GAT is an instance of the motif GGT that occurs in the first input string.

Zero or more (l, d) motifs are contained in any given set of input strings. Many of the known algorithms for PMS consider DNA strings for which $\Sigma = \{G, C, T, A\}$. There exist algorithms that deal with protein strings as well. The PMS problem is also known as the (l, d) -motif search (LDMS) problem.

Notation

The following mathematical notation is often used to describe PMS algorithms.

Assume that $S = \{s_1, s_2, s_3, \dots, s_n\}$ is the given set of input strings from an alphabet Σ . An l -mer of any string is nothing but a substring of the string of length l . Let $d_H(a, b)$ stand for the Hamming distance between any two l -mers a and b . Let a be an l -mer and s be an input string. Then, let $d_H(a, s)$ stand for the minimum Hamming distance between a and any l -mer b of s . If a is any l -mer and S is a set of input strings then let $d_H(a, S)$ stand for $\max_{s \in S} d_H(a, s)$. Let u be any l -mer. Then, the d -neighborhood of u , (denoted as $B_d(u)$), is nothing but the set of all the l -mers v such that $d_H(u, v) \leq d$. In other words, $B_d(u) = \{v: d_H(u, v) \leq d\}$. Refer to any such l -mer v as a d -neighbor of u . $B_d(x, y)$ is used to denote the common d -neighborhood of x and y , where x and y are two l -mers. $B_d(x, y)$ is nothing but the set of all l -mers that are within a distance of d from both x and y . Similarly, $B_d(x, y, z)$, etc. can be defined.

Algorithms

The scientific literature describes numerous algorithms for solving the PMS problem. These algorithms can be classified into two major types. Those algorithms that may not return the optimal answer(s) are referred to as approximation algorithms (or heuristic algorithms) and those that always return the optimal answer(s) are called exact algorithms.

Approximate

Examples of approximation (or heuristic) algorithms include Random Projection,^[2] PatternBranching,^[3] MULTIPROFILER,^[1] CONSENSUS,^[4] and ProfileBranching.^[3] These algorithms have been experimentally demonstrated to perform well.

Random projection

The algorithm^[2] is based on random projections. Let the motif M of interest be an l -mer and C be the collection of all the l -mers from all the n input strings. The algorithm projects these l -mers along k randomly chosen positions (for some appropriate value of k). The projection of each l -mer may be thought of as an integer. The projected values (which are k -mers) are grouped according to their integer values. In other words, hash all the l -mers using the k -mer of any l -mer as its hash value. All the l -mers that have the same hash value fall into the same hash bucket. Since the instances of any (l, d) motif are similar to each other, many of these instances will fall into the same bucket. Note that the Hamming

distance between any two instances of an (l, d) motif is no more than $2d$. The key idea of this algorithm is to examine those buckets that have a large number of l -mers in them. For each such bucket, an expectation maximization (EM) algorithm is used to check if an (l, d) motif can be found using the l -mers in the bucket.

Pattern branching

This algorithm^[3] is a local searching algorithm. If u is any l -mer, then there are $\binom{l}{d} 3^d$ l -mers that are d -neighbors of u , for DNA strings. This algorithm starts from each l -mer u in the input, searches the neighbors of u , scores them appropriately and outputs the best scoring neighbor.

Exact

Many exact algorithms are known for solving the PMS problem as well. Examples include the ones in (Martinez 1983),^[5] (Brazma, et al. 1998),^[6] (Galas, et al. 1985),^[7] (Sinha, et al. 2000),^[8] (Staden 1989),^[9] (Tomba 1999),^[10] (Helden, et al. 1998),^[11] (Rajasekaran, et al.),^[12] (Davila and Rajasekaran 2006),^[13] (Davila, Balla, and Rajasekaran 2006),^[14] Voting^[15] and RISOTTO.^[16]

WINNOWER and SP-STAR

The WINNOWER algorithm^[17] is a heuristic algorithm and it works as follows. If A and B are two instances of the same motif in two different input strings, then the Hamming distance between A and B is at most $2d$. It can be shown that the expected Hamming distance between A and B is $2d - \frac{4d^2}{3l}$.

WINNOWER constructs a collection C of all possible l -mers in the input. A graph $G(V, E)$ is constructed in which each l -mer of C will be a node. Two nodes u and v in G are connected by an edge if and only if the Hamming distance between u and v is at most $2d$ and they come from two different input strings.

If M is an (l, d) motif and if $M_1, M_2, \dots$, and M_n are instances of M in the input strings, then, clearly, these instances will form a clique in G . The WINNOWER algorithm has two phases. In the first phase, it identifies large cliques in G . In the second phase each such clique is examined to see if a motif can be extracted from this clique. Since the CLIQUE problem is intractable, WINNOWER uses a heuristic to solve CLIQUE. It iteratively constructs cliques of larger and larger sizes. If $N = mn$, then the run time of the algorithm is $O(N^{2d+1})$. This algorithm runs in a reasonable amount of time in practice especially for small values of d . Another algorithm called SP-STAR,^[17] is faster than WINNOWER and uses less memory. WINNOWER algorithm treats all the edges of G equally without distinguishing between edges based on similarities. SP-STAR scores the l -mers of C as well as the edges of G appropriately and hence eliminates more edges than WINNOWER per iteration.

(Bailey and Elkan, 1994)^[18] employs expectation maximization algorithms while Gibbs sampling is used by (Lawrence et al., 1993).^[19] MULTIPROFILER^[1] MEME,^[20] are also known PMS algorithms.

PMS series

In the last decade a series of algorithms with PMS as a prefix has been developed in the lab of Rajasekaran (<http://www.engr.uconn.edu/~rajasek/>). Some of these algorithms are described below.

PMS0

PMS0^[12] works as follows. Let $s_1, s_2, \dots, s_n$ be a given set of input strings each of length m . Let C be the collection of l -mers in s_1 . Let $C' = \cup_{u \in C} B_d(u)$. For each element v of C' check if it is a valid (l, d) -motif or not. Given an l -mer v , a check if it is a valid (l, d) -motif or not can be made in $O(mnl)$ time. Thus the run time of PMS0, assuming an alphabet of size 4, is $O(m^2nl \binom{l}{d} 3^d)$.

PMS1

This algorithm^[12] is based on radix sorting and has the following steps.

1. Generate the set of all l -mers in each input string. Let C_i correspond to the l -mers of s_i , for $1 \leq i \leq n$.
2. For each l -mer u in C_i ($1 < i < n$), generate $B_d(u)$. Let L_i be a collection of all of these neighbors (corresponding to all the l -mers of s_i).
3. Sort L_i (using radix sort) and eliminate any duplicates.
4. Compute $\bigcap_{i=1}^n L_i$. This can be done by merging the lists $L_1, L_2, \dots, L_n$. All the l -mers in this intersection are valid (l, d) motifs.

PMS2

Let the motif M of interest be of length l . If M occurs in every input string then any substring of M also occurs in every input string. Here occurrence means occurrence within a Hamming distance of d . It follows that there are at least $l-k+1$ strings each of length k (for $k \leq l$) such that each of these occurs in every input string.

Let Q be a collection of k -mers in M . Note that, in every input string s_i , there will be at least one position i_j such that a k -mer of Q occurs starting from i_j . Another k -mer of Q occurs starting from $i_j + 1$ and so on, with the last k -mer occurring at $i_j + l - k$. An l -mer can be obtained by combining these k -mers that occur starting from each such i_j .

PMS2^[12] works as follows. In the first phase find all the (k, d) motifs present in all the input strings (for some appropriate value of $k < l$). In the second phase, look for $(l-k+1)$ of these (k, d) motifs that occur starting from successive positions in each of the input strings. From every such collection of $(l-k+1)$ (k, d) -motifs, l -mer can be generated (if possible). Each such l -mer is a candidate (l, d) -motif. For each candidate motif, check if it is an (l, d) -motif or not in $O(mnl)$ time. This l -mer is returned as output if this is an (l, d) -motif.

PMS3

This algorithm^[12] enables one to handle large values of d . Let $d' = d/2$. Let M be the motif to be found with $|M| = l = 2l'$ for some integer l' . Let M_1 refer to the first half of M and M_2 be the next half. Let $s = a_1 a_2 \dots a_m$ be one of the input strings. M occurs in every input string. Let the occurrence of M (within a Hamming distance of d) in s start at position i . Let $s' = a_i a_{i+1} \dots a_{i+l'-1}$ and $s'' = a_{i+l'} \dots a_{i+l-1}$. It is clear that either the Hamming distance between M_1 and s' is at most d' or the Hamming distance between M_2 and s'' is at most d' . Either M_1 or M_2 occurs in every input string at a Hamming distance of at most d' . As a result, in at least n' strings (where $n' = n/2$) either M_1 or M_2 occurs with a Hamming distance of at

most d . The algorithm first obtains all the (l', d') -motifs that occur in at least $n/2$ of the input strings. It then uses these motifs and the above observations to identify all the (l, d) -motifs present in the input strings.

PMSPPrune

This algorithm introduces a tree structure for the motif candidates and uses a branch-and-bound algorithm to reduce the search space.^[21] Let $S = \{s_1, s_2, \dots, s_n\}$ be a given set of input strings. PMSPPrune follows the same strategy as PMS0: For every l -mer y in s_1 , it generates the set of neighbors of y and, for each of them, checks whether this is a motif or not. Some key steps in the algorithm are:

1. It generates the d -neighborhood of every l -mer y in s_1 using a tree of height d . The root of this tree will have y . Every l -mer that is at a distance of 1 from y will be a node in the tree that is at a distance of 1 from the root; every l -mer that is at a distance of 2 from y will be a node in the tree that is at a distance of 2 from the root; and so on. When a node in this tree is visited, check if the corresponding l -mer is an (l, d) -motif. I.e., if the l -mer is x , check if $d_H(x, S) \leq d$. If so, output this l -mer. In any case move to the next node in the tree. This tree is explored in a depth first manner.
2. If each node in the tree is visited for each l -mer y in s_1 , then the run time of PMSPPrune will be at least as much as that of PMS0. PMSPPrune uses some pruning conditions to prune subtrees that cannot possibly have any motifs in them.
3. For an l -mer x , which corresponds to a node in a subtree of height h , the algorithm uses the value of $d_H(x, S)$ and h to prune the descendants of x .
4. PMSPPrune calculates the value of $d_H(x, S)$ for the nodes (x) in the tree in an incremental way, taking into account the way in which the neighborhood is generated.

PMS4

PMS4^[22] is a technique that can be used to speedup any algorithm for the PMS problem. In many of the above algorithms there are two phases. In the first phase we come up with a set of candidate motifs and in the second phase check, for each candidate motif, if it is a valid (l, d) -motif. For each candidate motif it takes $O(mnl)$ time to check if it is a valid motif or not. PMS4 employs a similar two phase strategy. These phases are explained below. Let A be any PMS algorithm.

1. Run the algorithm A on k input strings (where $k < n$). An optimal value of k can be determined empirically. The k strings may be picked in a number of ways. For example, they could be the first k strings, random k strings, and so on. Let C be the collection of (l, d) -motifs found in these k strings. Clearly, C is a superset of the (l, d) -motifs present in the n given input strings.
2. for each l -mer v in C do

Check if v is a valid motif in $O(mnl)$ time. If so, output v .

PMS5 and PMS6

PMS5^[23] is an extension of PMS0. If $S = \{s_1, s_2, \dots, s_n\}$ is a set of strings (not necessarily of the same length), let $M_l^d(S)$ denote the (l, d) -motifs present in S . Let $S' = \{s_2, s_3, \dots, s_n\}$. PMS5 computes the (l, d) -motifs of S as $\bigcup_{L \in s_1} M_l^d(L, S')$. Here L refers to an l -mer.

One of the key steps in the algorithm is a subroutine to compute the common d -neighborhood of three l -mers. Let x, y, z be any three l -mers. To compute $B_d(x, y, z)$, PMS5 represents $B_d(x)$ as a tree $T_d(x)$. Each node in this tree represents an l -mer in $B_d(x)$. The root of $T_d(x)$ stands for the l -mer x . $T_d(x)$ has a depth of d . Nodes of $T_d(x)$ are traversed in a depth-first manner. The node and the l -mer it represents may be used interchangeably. While the tree is traversed, any node t will be output if t is in $B_d(y) \cap B_d(z)$. When any node t is visited, check if there is a descendent t' of t such that t' is in $B_d(y) \cap B_d(z)$. Prune the subtree rooted at t if there is no such descendent. In PMS5, the problem of checking if t has any descendent that is in $B_d(y) \cap B_d(z)$ is formulated as an integer linear program (ILP) on ten variables. This ILP is solved in $O(1)$ time. Solving the ILP instances is done as a preprocessing step and the results are stored in a lookup table.

Algorithm PMS6^[24] is an extension of PMS5 that improves the preprocessing step and also it uses efficient hashing techniques to store the lookup tables. As a result, it is typically faster than PMS5.

Shibdas Bandyopadhyay, Sartaj Sahni, Sanguthevar Rajasekaran, "PMS6: A fast algorithm for motif discovery," *iccabs*, pp.1-6, 2012 IEEE 2nd International Conference on Computational Advances in Bio and medical Sciences, 2012

qPMSPRune and qPMS7

Given a set $S = \{s_1, s_2, \dots, s_n\}$ of strings, and integers l, d , and q , an (l, d, q) -motif is defined to be a string M of length l that occurs in at least q of the n input strings within a Hamming distance of d . The qPMS (*Quorum Planted Motif Search*) problem is to find all the (l, d, q) -motifs present in the input strings. The qPMS problem captures the nature of motifs more precisely than the PMS problem does because, in practice, some motifs may not have motif instances in all of the input strings. Any algorithm for solving the qPMS problem (when $q \neq n$) is typically named with a prefix of 'q'. qPMSPRune is one of the first algorithms to address this version of the PMS problem.^[21] qPMSPRune exploits the following fact: If M is any (l, d, q) -motif of the input strings $s_1, s_2, \dots, s_n$, then there exists an i (with $1 \leq i \leq n - q + 1$) and an l -mer $x \in s_i$ such that M is in $B_d(x)$ and M is an $(l, d, q-1)$ -motif of the input strings excluding s_i . The algorithm processes every s_i , $1 \leq i \leq n$. While processing s_i , it considers every l -mer x of s_i . When considering x , it constructs $B_d(x)$ and identifies elements of $B_d(x)$ that are $(l, d, q-1)$ motifs (with respect to input strings other than s_i). $B_d(x)$ is represented as a tree with x as the root. This tree will be traversed in a depth first manner. The algorithm does not traverse the entire tree. Some of the subtrees are pruned using effective pruning conditions. In particular, a subtree is pruned if it can be inferred that none of the nodes in this subtree carries a motif of interest.

Algorithm qPMS7^[25] is an extension of qPMSPRune. Specifically, it is based on the following observation: If M is any (l, d, q) -motif of the input strings $s_1, s_2, \dots, s_n$, then there exist $1 \leq i \neq j \leq n$ and l -mer $x \in s_i$ and l -mer $y \in s_j$ such that M is in $B_d(x) \cap B_d(y)$ and M is an $(l, d, q-2)$ -motif of the input strings excluding s_i and s_j . The algorithm considers every possible pair (i, j) , $1 \leq i, j \leq n$ and $i \neq j$. For any pair (i, j) , every possible pair of l -mers (x, y) is considered (where x is from s_i and y is from s_j). While considering any x and y , the algorithm identifies all the elements of $B_d(x) \cap B_d(y)$ that are $(l, d, q-2)$ motifs (with respect to input strings other than s_i and s_j). An acyclic graph is used to represent and explore $B_d(x) \cap B_d(y)$. Call this graph $G_d(x, y)$. $G_d(x, y)$ is traversed in a depth first manner. Like in qPMSPRune, qPMS7 also employs some pruning conditions to prune subgraphs of $G_d(x, y)$.

RISOTTO

RISOTTO^[16] employs a suffix tree to identify the (l, d) -motifs. It is somewhat similar to PMS0. For every l -mer in s_I , it generates the d -neighborhood and for every l -mer in this neighborhood it walks through a suffix tree to check if this l -mer is an (l, d) -motif. Voting^[15] is similar to PMS1. Instead of using radix sorting, it uses hashing to compute L_i 's and their intersections.

Relative performance

PMS algorithms are typically tested on random benchmark data generated as follows: Twenty strings each of length 600 are generated randomly from the alphabet of interest. The motif M is also generated randomly and planted in each of the input strings within a Hamming distance of d . The motif instances are also generated randomly. Certain instances of the (l, d) -motif problem have been identified to be *challenging*. For a given value of l , the instance (l, d) is called challenging if d is the smallest integer for which the expected number of (l, d) -motifs that occur by random chance (in addition to the planted one) is one or more. For example, the following instances are challenging: (9, 2), (11, 3), (13, 4), (15, 5), (17, 6), (19, 7), etc. The performance of PMS algorithms is customarily shown only for challenging instances. Following is a table of time comparison of different PMS algorithms on the challenging instances of DNA sequences for the special case. This table is taken from the paper qPMS7.^[25] In this table several algorithms have been compared: qPMSP prune,^[21] qPMSP pruneI,^[25] Pampa,^[26] Voting,^[15] RISOTTO,^[16] PMS5,^[23] PMS6,^[24] qPMS7.^[25]

In the following table, the alphabet $\Sigma=\{A,C,G,T\}$, $n=20$, $m=600$, and $q=n=20$.

TIME COMPARISON OF DIFFERENT PMS ALGORITHMS

Algorithm	(13,4)	(15,5)	(17,6)	(19,7)	(21,8)	(23,9)
qPMS7	47 s	2.6 m	11 m	0.9 h	4.3 h	24 h
PMS6	67 s	3.2 m	14 m	1.16 h	5.8 h	-
PMS5	117 s	4.8 m	21.7 m	1.7 h	9.7 h	54
qPMSP pruneI	17 s	2.6 m	22.6 m	3.4 h	29 h	-
Pampa	35 s	6 m	40 m	4.8 h	-	-
qPMSP prune	45 s	10.2 m	78.7 m	15.2 h	-	-
Voting	104 s	21.6 m	-	-	-	-
RISOTTO	772 s	106 m	-	-	-	-

References

1. Keich, U.; Pevzner, P. A. (October 2002). "Finding motifs in the twilight zone". *Bioinformatics* **18** (10): 1374–1381. doi:10.1093/bioinformatics/18.10.1374 (https://dx.doi.org/10.1093%2Fbioinformatics%2F18.10.1374). PMID 12376382 (https://www.ncbi.nlm.nih.gov/pubmed/12376382).

2. Buhler, J.; Tompa, M. (2002). "Finding motifs using random projections". *J. Comput. Biol.* **9** (2): 225–242. doi:10.1089/10665270252935430 (https://dx.doi.org/10.1089%2F10665270252935430). PMID 12015879 (https://www.ncbi.nlm.nih.gov/pubmed/12015879).

3. Price, A.; Ramabhadran, S.; Pevzner, P. A. (October 2003). "Finding subtle motifs by branching from sample strings". *Bioinformatics*. 19 Suppl 2: ii149–55. doi:10.1093/bioinformatics/btg1072 (https://dx.doi.org/10.1093%2Fbioinformatics%2Fbtg1072) PMID 14534184

- (<https://www.ncbi.nlm.nih.gov/pubmed/14534184>).
4. Hertz, G. Z.; Stormo, G. D. (1999). "Identifying DNA and protein patterns with statistically significant alignments of multiple sequences". *Bioinformatics* **15** (7–8): 563–77. doi:10.1093/bioinformatics/15.7.563 (<https://dx.doi.org/10.1093%2Fbioinformatics%2F15.7.563>). PMID 10487864 (<https://www.ncbi.nlm.nih.gov/pubmed/10487864>).
5. Martinez, H. M. (July 1983). "An efficient method for finding repeats in molecular sequences" (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC326069>). *Nucleic Acids Res.* **11** (13): 4629–4634. doi:10.1093/nar/11.13.4629 (<https://dx.doi.org/10.1093%2Fnar%2F11.13.4629>). PMC 326069 (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC326069>). PMID 6866775 (<https://www.ncbi.nlm.nih.gov/pubmed/6866775>).
6. Brazma, A.; Jonassen, I.; Vilo, J.; Ukkonen, E. (November 1998). "Predicting gene regulatory elements in silico on a genomic scale" (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC310790>). *Genome Res.* **8** (11): 1202–1215. doi:10.1101/gr.8.11.1202 (<https://dx.doi.org/10.1101%2Fgr.8.11.1202>). PMC 310790 (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC310790>). PMID 9847082 (<https://www.ncbi.nlm.nih.gov/pubmed/9847082>).
7. Galas, D. J.; Eggert, M.; Waterman, M. S. (November 1985). "Rigorous pattern-recognition methods for DNA sequences. Analysis of promoter sequences from *Escherichia coli*". *J. Mol. Biol.* **186** (1): 117–128. doi:10.1016/0022-2836(85)90262-1 (<https://dx.doi.org/10.1016%2F0022-2836%2885%2990262-1>). PMID 3908689 (<https://www.ncbi.nlm.nih.gov/pubmed/3908689>).
8. Sinha, S.; Tompa, M. (2000). "A statistical method for finding transcription factor binding sites". *Proc Int Conf Intell Syst Mol Biol* **8**: 344–354. PMID 10977095 (<https://www.ncbi.nlm.nih.gov/pubmed/10977095>).
9. Staden, R. (October 1989). "Methods for discovering novel motifs in nucleic acid sequences". *Comput. Appl. Biosci.* **5** (4): 293–8. doi:10.1093/bioinformatics/5.4.293 (<https://dx.doi.org/10.1093%2Fbioinformatics%2F5.4.293>). PMID 2684350 (<https://www.ncbi.nlm.nih.gov/pubmed/2684350>).
10. Tompa, M. (1999). "An exact method for finding short motifs in sequences, with application to the ribosome binding site problem". *Proc Int Conf Intell Syst Mol Biol*: 262–271. PMID 10786309 (<https://www.ncbi.nlm.nih.gov/pubmed/10786309>).
11. van Helden, J.; André, B.; Collado-Vides, J. (September 1998). "Extracting regulatory sites from the upstream region of yeast genes by computational analysis of oligonucleotide frequencies". *J. Mol. Biol.* **281** (5): 827–842. doi:10.1006/jmbi.1998.1947 (<https://dx.doi.org/10.1006%2Fjmbi.1998.1947>). PMID 9719638 (<https://www.ncbi.nlm.nih.gov/pubmed/9719638>).
12. Rajasekaran, S.; Balla, S.; Huang, C. H. (October 2005). "Exact algorithms for planted motif problems". *J. Comput. Biol.* **12** (8): 1117–1128. doi:10.1089/cmb.2005.12.1117 (<https://dx.doi.org/10.1089%2Fcmb.2005.12.1117>). PMID 16241901 (<https://www.ncbi.nlm.nih.gov/pubmed/16241901>).
13. Davila, J.; Rajasekaran, S. (2006). "Extending Pattern Branching to Handle Challenging Instances". *Bioinformatics and BioEngineering*: 65–69. doi:10.1109/BIBE.2006.253317 (<https://dx.doi.org/10.1109%2FBIBE.2006.253317>). ISBN 0-7695-2727-2.
14. Davila, J.; Balla, S.; Rajasekaran, S (2006). "Space and time efficient algorithms for planted motif search". *Proc. 6th International Conference on Computational Science (ICCS 2006)/ 2nd International Workshop on Bioinformatics Research and Applications (IWBRA 2006) LNCS 3992*. CiteSeerX: 10.1.1.94.4572 (<http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.94.4572>).
15. Chin, F. Y. L.; Leung, H. C. M. (2005). "Voting algorithms for discovering long motifs". *Proceedings of the Third Asia-Pacific Bioinformatics Conference (APBC)*: 261–271. doi:10.1142/9781860947322_0026 (https://dx.doi.org/10.1142%2F9781860947322_0026). ISBN 978-1-86094-477-2.
16. Pisanti, N.; Carvalho, A.; Marsan, L.; Sagot, M. F. (2006). "Risotto: Fast extraction of motifs with mismatches". *Proceedings of the 7th Latin American Theoretical Informatics Symposium*: 757–768. CiteSeerX: 10.1.1.60.1028 (<http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.60.1028>).
17. Pevzner, P. A.; Sze, S. H. (2000). "Combinatorial approaches to finding subtle signals in DNA sequences". *Proc Int Conf Intell Syst Mol Biol* **8**: 269–278. PMID 10977088 (<https://www.ncbi.nlm.nih.gov/pubmed/10977088>).
18. Bailey, T. L.; Elkan, C. (1994). "Fitting a mixture model by expectation maximization to discover motifs in biopolymers". *Proc Int Conf Intell Syst Mol Biol* **2**: 28–36. PMID 7584402 (<https://www.ncbi.nlm.nih.gov/pubmed/7584402>).
19. Lawrence, C. E.; Altschul, S. F.; Boguski, M. S.; Liu, J. S.; Neuwald, A. F.; Wootton, J. C. (October 1993). "Detecting subtle sequence signals: a Gibbs sampling strategy for multiple alignment". *Science* **262** (5131): 201–204. doi:10.1126/science.2011126 (<https://doi.org/10.1126/science.2011126>). PMID 8211126

- 208–214. doi:10.1126/science.8211139 (<https://dx.doi.org/10.1126%2Fscience.8211139>). PMID 8211139 (<https://www.ncbi.nlm.nih.gov/pubmed/8211139>).
20. Bailey, T. L.; Elkan, Charles (January 1995). "Unsupervised learning of multiple motifs in biopolymers using expectation maximization". *Machine Learning* **21** (1–2): 51–80. doi:10.1007/BF00993379 (<https://dx.doi.org/10.1007%2FBF00993379>).
 21. Davila, J.; Balla, S.; Rajasekaran, S. (2007). "Fast and practical algorithms for planted (l, d) motif search". *IEEE/ACM Trans Comput Biol Bioinform* **4** (4): 544–552. doi:10.1109/TCBB.2007.70241 (<https://dx.doi.org/10.1109%2FTCBB.2007.70241>). PMID 17975266 (<https://www.ncbi.nlm.nih.gov/pubmed/17975266>).
 22. Rajasekaran, S.; Dinh, H. (2011). "A speedup technique for (l, d)-motif finding algorithms" (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3063805>). *BMC Res Notes* **4**: 54. doi:10.1186/1756-0500-4-54 (<https://dx.doi.org/10.1186%2F1756-0500-4-54>). PMC 3063805 (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3063805>). PMID 21385438 (<https://www.ncbi.nlm.nih.gov/pubmed/21385438>).
 23. Dinh, H.; Rajasekaran, S.; Kundeti, V. K. (2011). "PMS5: an efficient exact algorithm for the (ℓ, d)-motif finding problem" (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3269969>). *BMC Bioinformatics* **12**: 410. doi:10.1186/1471-2105-12-410 (<https://dx.doi.org/10.1186%2F1471-2105-12-410>). PMC 3269969 (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3269969>). PMID 22024209 (<https://www.ncbi.nlm.nih.gov/pubmed/22024209>).
 24. Bandyopadhyay, S.; Sahni, S.; Rajasekaran, S. (2012). "PMS6: A fast algorithm for motif discovery". *IEEE 2nd International Conference on Computational Advances in Bio and medical Sciences*: 1. doi:10.1109/ICCABS.2012.6182627 (<https://dx.doi.org/10.1109%2FICCABS.2012.6182627>). ISBN 978-1-4673-1321-6.
 25. Dinh, H.; Rajasekaran, S.; Davila, J. (2012). Brusic, Vladimir, ed. "qPMS7: a fast algorithm for finding (ℓ, d)-motifs in DNA and protein sequences" (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3404135>). *PLoS ONE* **7** (7): e41425. doi:10.1371/journal.pone.0041425 (<https://dx.doi.org/10.1371%2Fjournal.pone.0041425>). PMC 3404135 (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3404135>). PMID 22848493 (<https://www.ncbi.nlm.nih.gov/pubmed/22848493>).
 26. Davila, J.; Balla, S.; Rajasekaran, S. (2007). "Pampa: An improved branch and bound algorithm for planted (l, d) motif search". *Technical Report*. CiteSeerX: 10.1.1.93.6500 (<http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.93.6500>).

External links

- Rajasekaran, S.; Dinh, H. "PMS Motif Search" (<http://motifsearch.com>). University of Connecticut.
- Rajasekaran, S.; Dinh, H. "Panoptic Motif Search" (<http://pms.engr.uconn.edu>). University of Connecticut.

Retrieved from "http://en.wikipedia.org/w/index.php?title=Planted_motif_search&oldid=662671468"

Categories: Bioinformatics | Computational biology

-
- This page was last modified on 16 May 2015, at 23:07.
 - Text is available under the Creative Commons Attribution-ShareAlike License; additional terms may apply. By using this site, you agree to the Terms of Use and Privacy Policy. Wikipedia® is a registered trademark of the Wikimedia Foundation, Inc., a non-profit organization.